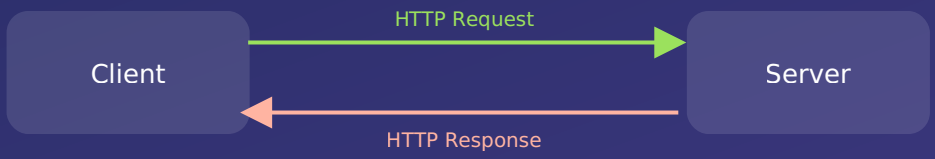
BACKEND ENGINEERING NOTES

HTTP & Backend Fundamentals

A complete, plain-language guide with real Node.js & Spring Boot examples, diagrams, and interview-ready summaries.



Expanded edition — with real backend code, diagrams & deeper explanations

Table of Contents

1. What Is HTTP, Really? 3
2. The Client–Server Model 3
3. Statelessness & Authentication Tokens 4
4. Anatomy of an HTTP Request 5
5. Anatomy of an HTTP Response 6
6. HTTP Methods, Explained with a Real API 7
7. Idempotency, In Depth 8
8. HTTP Headers, One by One 9
9. CORS — Why It Exists & How It Works 10
10. CORS Preflight (OPTIONS) 11
11. HTTP Status Codes — Full Reference 12
12. Status Code Pairs People Confuse 13
13. HTTP Caching Explained 14
14. Cache-Control, ETag, Last-Modified 15
15. 304 Not Modified — Full Flow 16
16. Content Negotiation 16
17. Compression (gzip/br) 17
18. Persistent Connections / Keep-Alive 17
19. Multipart/Form-Data (File Uploads) 18
20. Streaming Large Responses 19
21. SSL, TLS & HTTPS 20
22. HTTP/1.0 → HTTP/3 Timeline 20
23. The Complete Request Lifecycle in a Real Backend 21
24. Interview Memorization Sheet 22
25. One-Minute Revision 23

1 What Is HTTP, Really?

HTTP (HyperText Transfer Protocol) is the set of rules that lets a client and a server talk to each other over a network. Every time you open a website, tap a button in an app, or call an API, HTTP is the language carrying that conversation.

Think of it like ordering food at a counter. You (the **client**) walk up and say exactly what you want, in a format the cashier understands ("one burger, no onions"). The cashier (the **server**) processes the order and hands back a result ("here's your burger" or "sorry, we're out"). HTTP standardizes that exchange so that any client (a browser, a mobile app, a script) can talk to any server (Node.js, Spring Boot, Django, etc.) without needing to know how the other side is built internally.

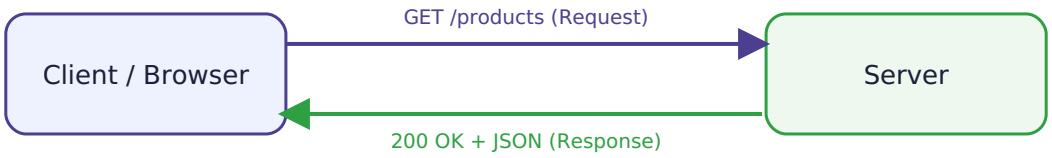


Fig 1.1 — The basic HTTP request/response cycle.

Real example

When you visit an online store's product page, your browser sends `GET /products/42` to the store's server. The server queries its database for product #42 and sends back an HTML page or a JSON object describing that product. That single exchange — one request, one response — is the fundamental unit of HTTP.

Why backend developers must understand HTTP deeply

Every backend framework — Express (Node.js), Spring Boot (Java), Django (Python), Rails (Ruby) — is ultimately just a structured way of receiving HTTP requests and producing HTTP responses. If you understand HTTP itself, you can pick up any backend framework quickly, because they all map onto the same underlying concepts: routes, methods, headers, status codes, and bodies.

2 The Client–Server Model

The client always starts the conversation. The server only ever responds — it never contacts the client on its own initiative in plain HTTP (that's what WebSockets or Server-Sent Events are for).

Role	Who plays it	Responsibility
Client	Browser, mobile app, Postman, another backend service, a frontend built in React/Angular	Initiates the request: chooses the method, URL, headers, and body
Server	Node.js/Express app, Spring Boot app, a load balancer, a CDN	Receives the request, authenticates it, runs business logic, talks to the database, and returns a response

Real example — a login flow

Your React frontend (the client) sends `POST /api/auth/login` with a JSON body containing an email and password. Your Express or Spring Boot backend (the server) checks the credentials against the database, generates a JWT token, and responds with `200 OK` and that token in the body. The frontend then stores the token and attaches it to every future request.

Express.js — a minimal server that plays the "server" role

NODE.JS / EXPRESS

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/products/:id', (req, res) => {
  const product = db.findProductById(req.params.id);
  if (!product) return res.status(404).json({ error: 'Not found' });
  res.status(200).json(product);
});

app.listen(3000, () => console.log('Server listening on port 3000'));
```

Spring Boot — the same idea in Java

JAVA / SPRING BOOT

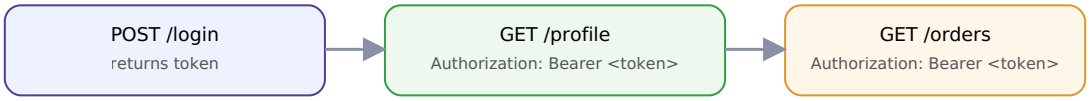
```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProduct(@PathVariable Long id) {
        Product product = productService.findById(id);
        if (product == null) {
            return ResponseEntity.status(404).build();
        }
        return ResponseEntity.ok(product);
    }
}
```

3 Statelessness & Authentication Tokens

HTTP is **stateless**: the server does not remember anything about previous requests just because they came from the same client. Every single request must carry all the information the server needs to understand and process it.

Each request must carry its own identity — the server keeps no memory between calls:



Without the token attached, request 2 and 3 would look identical to a brand-new anonymous visitor.

Fig 3.1 — Statelessness in practice: the token, not server memory, carries identity.

Real example: a JWT-based session

NODE.JS / EXPRESS — ATTACHING AND VERIFYING A TOKEN

```
// Step 1: client logs in, server issues a JWT
app.post('/login', (req, res) => {
  const user = authenticate(req.body.email, req.body.password);
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn: '1h' });
  res.json({ token });
});

// Step 2: every later request must include that token
app.get('/orders', requireAuth, (req, res) => {
  const orders = db.findOrdersByUser(req.userId); // req.userId came from the token
  res.json(orders);
});

function requireAuth(req, res, next) {
  const header = req.headers.authorization;         // "Bearer eyJhbGciOi..."
  if (!header) return res.status(401).json({ error: 'Missing token' });
  const token = header.split(' ')[1];
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = payload.userId;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

Why statelessness matters for scaling

Because no server needs to "remember" a client, any request can be routed to any server behind a load balancer. If Server A handled your login and Server B handles your next request, it doesn't matter — as long as your token travels with you, Server B can verify it independently. This is exactly why large systems can horizontally scale to hundreds of stateless backend instances.

4 Anatomy of an HTTP Request

Every HTTP request is made of four parts: a **method**, a **URL**, **headers**, and an optional **body**.

The four parts

- ✓ **Method** — the action: GET, POST, PUT, PATCH, DELETE...
- ✓ **URL** — which resource, e.g. `/users/42`
- ✓ **Headers** — metadata like content type or auth token
- ✓ **Body** — the actual data payload (not present on GET)

RAW HTTP REQUEST

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{"name":"Krrish","age":22}
```

The same request as real backend code

WHAT THE CLIENT SENDS (FETCH, IN A REACT APP)

```
const res = await fetch('https://api.example.com/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  },
  body: JSON.stringify({ name: 'Krrish', age: 22 })
});
const data = await res.json();
```

WHAT THE SERVER READS (EXPRESS)

```
app.post('/users', requireAuth, (req, res) => {
  console.log(req.method);            // "POST"
  console.log(req.path);              // "/users"
  console.log(req.headers['content-type']); // "application/json"
  console.log(req.body);              // { name: 'Krrish', age: 22 }

  const newUser = db.createUser(req.body);
  res.status(201).json(newUser);
});
```

Common mistake

Forgetting `Content-Type: application/json` is one of the most common bugs beginners hit — many frameworks (including Express with `express.json()`) will silently leave `req.body` empty or undefined if that header is missing, because they don't know how to parse the raw bytes.

5 Anatomy of an HTTP Response

A response is the mirror image of a request: it has a **status code**, **headers**, and an optional **body**.

RAW HTTP RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /users/103

{"id":103,"name":"Krrish","age":22}
```

THE SAME RESPONSE BUILT IN SPRING BOOT

```
@PostMapping("/users")
public ResponseEntity<User> createUser(@RequestBody User newUser) {
    User saved = userService.save(newUser);
    URI location = URI.create("/users/" + saved.getId());
    return ResponseEntity.created(location).body(saved); // 201 Created
}
```

Notice the pattern: a status code tells the client *what kind of outcome* happened (success, client error, server error), headers describe the response's metadata, and the body carries the actual data — often JSON in modern APIs, but it could also be HTML, plain text, a file, or nothing at all.

6 HTTP Methods, Explained with a Real API

Methods describe the *intent* of a request. Below is a single resource — `/notes/:id` — implemented with every method, so you can see how the same URL behaves differently depending on the verb.

Method	Meaning	Typical use	Has a body?
GET	Read	Fetch a note or list of notes	No
POST	Create	Create a brand-new note	Yes
PUT	Replace	Overwrite a note completely	Yes
PATCH	Partial update	Change just the title, for example	Yes
DELETE	Remove	Delete a note	Usually no

EXPRESS — THE FULL CRUD RESOURCE

```
const router = express.Router();

router.get('/notes/:id', (req, res) => {
  const note = db.findNote(req.params.id);
  note ? res.json(note) : res.status(404).end();
});

router.post('/notes', (req, res) => {
  const note = db.createNote(req.body);         // { title, content }
  res.status(201).json(note);
});

router.put('/notes/:id', (req, res) => {
  // PUT replaces the WHOLE note — client must send every field
  const note = db.replaceNote(req.params.id, req.body);
  res.json(note);
});

router.patch('/notes/:id', (req, res) => {
  // PATCH only changes the fields that were sent
  const note = db.updateNoteFields(req.params.id, req.body); // e.g. { title: "New title" }
  res.json(note);
});

router.delete('/notes/:id', (req, res) => {
  db.deleteNote(req.params.id);
  res.status(204).end();                        // 204 = success, no body
});
```

PUT vs PATCH in one sentence

`PUT` says "here is the complete new version of this resource, replace everything" — if you omit a field, many implementations will wipe it out. `PATCH` says "only change the fields I mention" — everything else stays as it was.

7 Idempotency, In Depth

An operation is **idempotent** if performing it once has the same effect on the server's state as performing it many times.

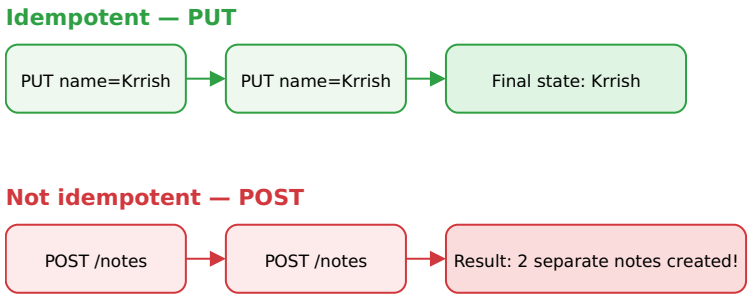


Fig 7.1 — Repeating PUT leaves the same final state; repeating POST creates duplicates.

Real-world danger: double-clicking "Pay Now"

Imagine a checkout button that triggers `POST /payments`. If a user's network is slow and they click twice, and the backend isn't careful, two separate charges could be created — because POST is not idempotent by default. This is why payment APIs (like Stripe) support an **idempotency key** header: the client generates a unique key per attempt, and the server stores which keys it has already processed, refusing to charge twice for the same key.

EXPRESS — IMPLEMENTING AN IDEMPOTENCY KEY FOR A PAYMENT POST

```
app.post('/payments', async (req, res) => {
  const idempotencyKey = req.headers['idempotency-key'];
  const existing = await db.findPaymentByKey(idempotencyKey);
  if (existing) {
    return res.status(200).json(existing); // already processed — return the same result
  }
  const payment = await chargeCard(req.body);
  await db.savePaymentWithKey(idempotencyKey, payment);
  res.status(201).json(payment);
});
```

8 HTTP Headers, One by One

Headers are key-value pairs of metadata that travel alongside a request or response. They don't carry the "main" data — they describe it.

Header	Direction	What it does	Real example
`Content-Type`	Both	Format of the body	`application/json`
`Authorization`	Request	Carries credentials	`Bearer eyJhbGciOi...`
`Accept`	Request	Preferred response format	`application/json`
`Content-Length`	Response	Body size in bytes	`348`
`Cache-Control`	Response	Caching rules	`max-age=3600`
`ETag`	Response	Version fingerprint of the resource	`"33a64df551"`
`User-Agent`	Request	Identifies the client software	`Mozilla/5.0 ...`

READING AND SETTING HEADERS IN EXPRESS

```
app.get('/report', (req, res) => {
  console.log(req.get('Accept'));           // what the client wants back
  console.log(req.get('User-Agent'));       // which client/browser is calling

  res.set('Cache-Control', 'max-age=3600'); // tell the client to cache for 1 hour
  res.set('Content-Type', 'application/json');
  res.json({ generatedAt: new Date() });
});
```

9 CORS — Why It Exists & How It Works

CORS (Cross-Origin Resource Sharing) is a **browser** security rule. It stops a web page loaded from one origin from silently reading responses from a different origin, unless that origin explicitly allows it.



Fig 9.1 — Two different ports on the same machine still count as different origins.

> **The classic "it works in Postman but not in the browser" bug**
>
> Postman doesn't enforce CORS — it's not a browser. So a request that fails in your React app with a CORS error in the console might work perfectly in Postman. This confuses beginners constantly: the fix isn't in the frontend, it's in the backend's CORS configuration.

ENABLING CORS IN EXPRESS

```
const cors = require('cors');

app.use(cors({
  origin: 'http://localhost:5173',        // only allow this frontend origin
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

ENABLING CORS IN SPRING BOOT

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("Content-Type", "Authorization");
    }
}
```

10 CORS Preflight (OPTIONS)

For requests that aren't considered "simple" (e.g. using PUT/DELETE, a custom header like `Authorization`, or a JSON content type), the browser first sends a lightweight `OPTIONS` request to ask permission before sending the real one.

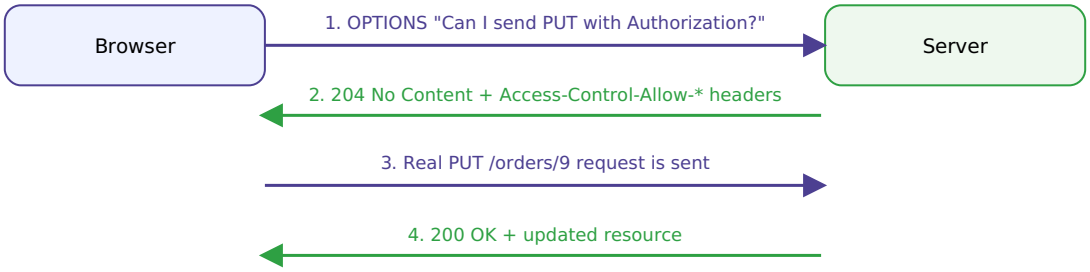


Fig 10.1 — Preflight happens automatically before "non-simple" cross-origin requests.

WHAT THE PREFLIGHT RESPONSE HEADERS LOOK LIKE

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Max-Age: 86400
```

`Access-Control-Max-Age` tells the browser how long (in seconds) it can cache this preflight answer, so it doesn't have to re-ask for every single request.

11 HTTP Status Codes — Full Reference

The first digit of a status code tells you the category at a glance: **2xx** success, **3xx** redirect, **4xx** the client made a mistake, **5xx** the server made a mistake.

200 OK	Request succeeded, body included
201 Created	New resource created (typical for POST)
204 No Content	Success, no body (typical for DELETE)
301 Moved Permanently	Resource permanently lives elsewhere now
302 Found	Temporary redirect
304 Not Modified	Client's cached copy is still valid
400 Bad Request	Malformed or invalid request data
401 Unauthorized	Missing or invalid authentication
403 Forbidden	Authenticated, but not allowed to do this
404 Not Found	Resource doesn't exist
405 Method Not Allowed	URL exists, but not for this HTTP method
409 Conflict	Conflicts with the current server state
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Unhandled exception in the backend
502 Bad Gateway	Upstream service returned garbage/nothing usable
503 Service Unavailable	Server temporarily can't handle requests (overload, maintenance)
504 Gateway Timeout	Upstream service never responded in time

A REAL EXPRESS ERROR-HANDLING PATTERN THAT USES SEVERAL OF THESE

```
app.post('/orders', requireAuth, (req, res) => {
  if (!req.body.productId) {
    return res.status(400).json({ error: 'productId is required' });   // 400
  }
  const product = db.findProduct(req.body.productId);
  if (!product) {
    return res.status(404).json({ error: 'Product not found' });       // 404
  }
  if (product.stock <= 0) {
    return res.status(409).json({ error: 'Product out of stock' });    // 409
  }
  try {
    const order = db.createOrder(req.userId, product);
    res.status(201).json(order);                                       // 201
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Something went wrong' });           // 500
  }
});
```

12 Status Code Pairs People Confuse

401 vs 403

401 Unauthorized really means "un-*authenticated*" — the server doesn't know who you are (missing/invalid token). **403 Forbidden** means the server knows exactly who you are, but you're not allowed to perform this action (e.g. a regular user trying to hit an admin-only route).

EXPRESS — BOTH IN ONE ROUTE

```
app.delete('/admin/users/:id', (req, res) => {
  if (!req.userId) return res.status(401).json({ error: 'Please log in' });     // who ARE you?
  if (!req.userIsAdmin) return res.status(403).json({ error: 'Admins only' });  // I know you, but no.
  db.deleteUser(req.params.id);
  res.status(204).end();
});
```

400 vs 404

400 means the request itself is broken — bad JSON, a missing required field. **404** means the request was perfectly well-formed, but the thing it's asking for doesn't exist.

500 vs 503 vs 504

500 is a bug or unhandled exception inside your own code. **503** means the server is intentionally or temporarily unable to serve traffic (overloaded, deploying, in maintenance mode). **504** specifically means *your server was waiting on another service* (a database, a third-party API) and that upstream service never answered in time.

13 HTTP Caching Explained

Caching means storing a response so it can be reused later instead of asking the server to regenerate (and re-send) the exact same thing again.

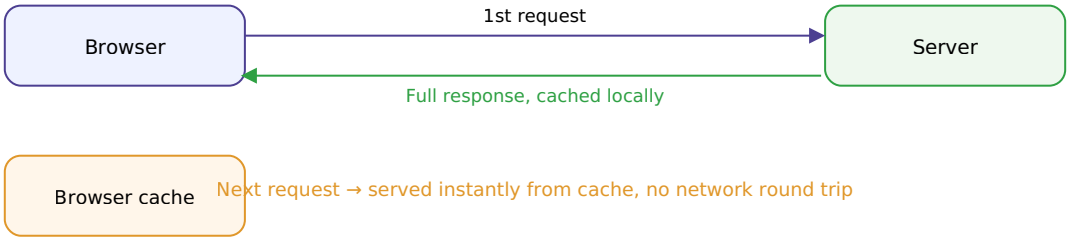


Fig 13.1 — A cached response skips the network entirely until it expires.

Real-world impact: serving a cached product-image or a static JS bundle from cache means the user gets an instant response and the server handles far less load — this is why CDNs like Cloudflare exist, essentially as giant distributed caches sitting between users and origin servers.

14 Cache-Control, ETag, Last-Modified

Header	What it tells the client	Example
`Cache-Control`	How long the response can be reused, and whether it can be shared	`max-age=3600, public`
`ETag`	A fingerprint/hash of the current version of the resource	`"5d8c72a4b9"`
`Last-Modified`	The timestamp the resource was last changed	`Wed, 21 Oct 2025 07:28:00 GMT`

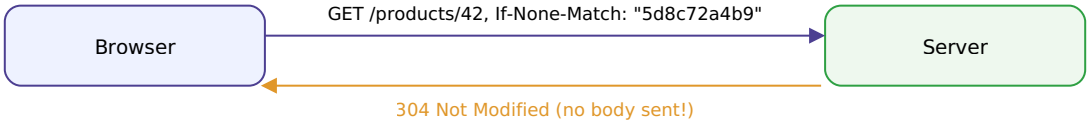
EXPRESS — SENDING CACHING HEADERS FOR A STATIC-ISH API RESOURCE

```
app.get('/products/:id', (req, res) => {
  const product = db.findProduct(req.params.id);
  const etag = generateHash(product);

  res.set('ETag', etag);
  res.set('Cache-Control', 'max-age=300'); // reusable for 5 minutes
  res.json(product);
});
```

15 304 Not Modified — Full Flow

When a cached response's freshness window (`max-age`) expires, the browser doesn't just blindly re-download — it can ask "has this actually changed?" using the stored ETag.



If the resource HAD changed, the server would instead answer 200 OK with the new body + a new ETag.

Fig 15.1 — 304 saves bandwidth: the body is never re-sent if nothing changed.

EXPRESS — HONORING IF-NONE-MATCH

```
app.get('/products/:id', (req, res) => {
  const product = db.findProduct(req.params.id);
  const currentEtag = generateHash(product);

  if (req.headers['if-none-match'] === currentEtag) {
    return res.status(304).end();  // tell browser: your cache is still good
  }
  res.set('ETag', currentEtag);
  res.json(product);
});
```

16 Content Negotiation

A single URL can return different representations depending on what the client says it prefers.

```
Accept: application/json
Accept-Language: en
Accept-Encoding: gzip
```

EXPRESS — BRANCHING ON ACCEPT

```
app.get('/report/:id', (req, res) => {
  const report = db.findReport(req.params.id);
  res.format({
    json: () => res.json(report),
    html: () => res.send(renderHtml(report)),
    default: () => res.status(406).send('Not Acceptable')
  });
});
```

17 Compression (gzip / brotli)

Compression shrinks a response before it's sent over the network, then the client decompresses it back to the original bytes.



Fig 17.1 — Compression can shrink a JSON payload dramatically before it hits the wire.

EXPRESS — ENABLING GZIP FOR EVERY RESPONSE

```
const compression = require('compression');
app.use(compression()); // automatically compresses responses when the client supports it
```

18 Persistent Connections / Keep-Alive

HTTP/1.0 opened a brand-new TCP connection per request — slow, because of the repeated handshake cost. HTTP/1.1 introduced **keep-alive**: one TCP connection can carry many request/response cycles before closing.

```
Connection: keep-alive

One TCP connection:
  Request 1  →  Response 1
  Request 2  →  Response 2
  Request 3  →  Response 3
  (connection closes only when idle too long, or explicitly closed)
```

This is one reason why loading a page with 40 small images is much faster than it would otherwise be — the browser reuses a handful of connections instead of opening 40 new ones.

19 Multipart/Form-Data (File Uploads)

`multipart/form-data` lets a single request carry several distinct "parts" — some text fields and one or more binary files — each separated by a boundary string.

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryX

------WebKitFormBoundaryX
Content-Disposition: form-data; name="username"

krrish
------WebKitFormBoundaryX
Content-Disposition: form-data; name="avatar"; filename="photo.png"
Content-Type: image/png

<binary image bytes>
------WebKitFormBoundaryX--
```

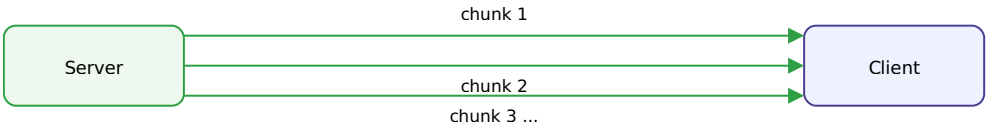
EXPRESS — HANDLING A FILE UPLOAD WITH MULTER

```
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('avatar'), (req, res) => {
  console.log(req.body.username);   // "krrish"
  console.log(req.file.filename);   // saved file name on disk
  res.status(201).json({ url: `/uploads/${req.file.filename}` });
});
```

20 Streaming Large Responses

Instead of waiting to assemble the entire response before sending anything, the server can stream data out in chunks as it becomes available — useful for large files, live logs, or AI chat responses.



Client assembles/renders each chunk as it arrives — no need to wait for everything.

Fig 20.1 — Chunked transfer: the client starts consuming data before the response fully finishes.

EXPRESS — A SIMPLE SERVER-SENT EVENTS (SSE) STREAM

```
app.get('/events', (req, res) => {
  res.set({
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive'
  });

  const interval = setInterval(() => {
    res.write(`data: ${JSON.stringify({ time: Date.now() })}\n\n`);
  }, 1000);

  req.on('close', () => clearInterval(interval)); // clean up when client disconnects
});
```

21 SSL, TLS & HTTPS

SSL was the original protocol for encrypting traffic; it's outdated and considered insecure today. TLS (Transport Layer Security) is its modern replacement. **HTTPS is simply HTTP running over a TLS-encrypted connection.**

```
HTTP + TLS = HTTPS

Without TLS: anyone on the network path can read your password in plain text.
With TLS:    the data is encrypted end-to-end between client and server.
```

What TLS actually gives you

Three things: **encryption** (nobody in the middle can read the data), **integrity** (nobody can silently tamper with it), and **authentication** (a certificate proves you're really talking to the real server, not an impostor).

22 HTTP/1.0 → HTTP/3 Timeline

Version	Key idea to remember
HTTP/1.0	New TCP connection per request — slow
HTTP/1.1	Persistent connections (keep-alive) became the default
HTTP/2	Binary framing + multiplexing — many requests share one connection with no head-of-line blocking at the HTTP layer
HTTP/3	Runs over QUIC (built on UDP) instead of TCP — faster connection setup, better on unstable networks

23 The Complete Request Lifecycle in a Real Backend

Here is everything from this guide, tied together in one realistic request through a layered backend.

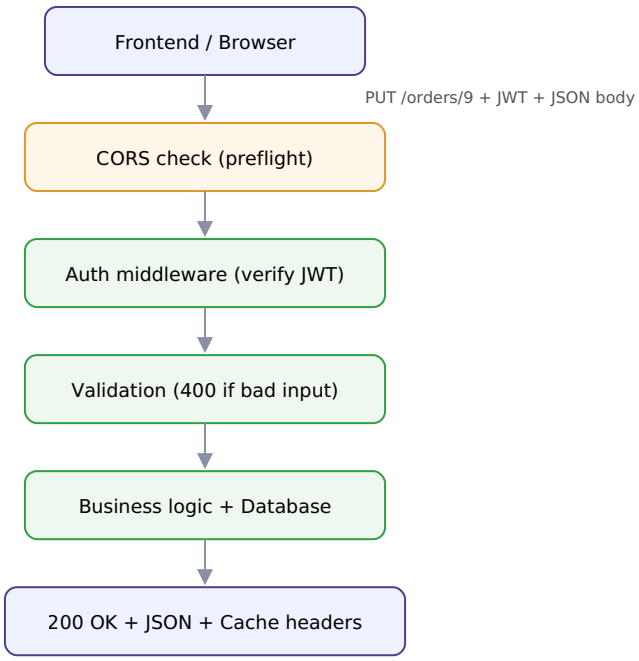


Fig 23.1 — Everything covered in this guide, applied to one real request end to end.

EXPRESS — THE SAME LIFECYCLE AS ACTUAL MIDDLEWARE

```
app.use(cors({ origin: 'http://localhost:5173' }));   // 1. CORS
app.use(express.json());

app.put('/orders/:id',
  requireAuth,                                        // 2. Authentication (401)
  validateOrderBody,                                   // 3. Validation (400)
  async (req, res) => {                                // 4. Business logic
    const order = await db.findOrder(req.params.id);
    if (!order) return res.status(404).json({ error: 'Not found' });
    if (order.userId !== req.userId) return res.status(403).json({ error: 'Forbidden' });

    const updated = await db.replaceOrder(req.params.id, req.body);
    res.set('Cache-Control', 'no-store');              // 5. Response headers
    res.status(200).json(updated);                      // 6. Response body
  }
);
```

24 Interview Memorization Sheet

- ✓ HTTP is stateless — the server remembers nothing between requests on its own.
- ✓ Request = method + URL + headers + optional body.
- ✓ Response = status code + headers + optional body.
- ✓ GET = read, POST = create, PUT = replace, PATCH = partial update, DELETE = remove.
- ✓ PUT and DELETE are idempotent; POST is not.
- ✓ 401 = "who are you?" (authentication); 403 = "I know you, but no" (authorization).
- ✓ 400 = the request itself is invalid; 404 = the resource doesn't exist.
- ✓ 200 = success; 201 = created; 204 = success, no body.
- ✓ 500 = bug in your own server; 503 = server temporarily overloaded/down; 504 = an upstream service timed out.
- ✓ CORS is a browser-only security mechanism that blocks cross-origin responses unless explicitly allowed.
- ✓ OPTIONS is used for CORS preflight on "non-simple" requests.
- ✓ 304 means "your cached copy is still valid" — no body is resent.
- ✓ Cache-Control, ETag, and Last-Modified drive HTTP caching decisions.
- ✓ HTTPS = HTTP + TLS encryption, integrity, and server authentication.
- ✓ multipart/form-data is the standard way to upload files alongside form fields.

25 One-Minute Revision

HTTP: the language client and server speak.
Request: "what do I want?" — method, URL, headers, body.
Response: "what happened?" — status, headers, body.
Statelessness: every request stands alone; tokens carry identity.
CORS: the browser's cross-origin permission system.
Caching: reuse old data instead of re-fetching it.
Compression: shrink data before sending it.
Multipart: send files and fields together.
Streaming: send large data piece by piece.
HTTPS: HTTP, but encrypted and authenticated via TLS.

Expanded from the original simplified HTTP/backend notes, with additional explanations, real Node.js/Express and Java/Spring Boot examples, and original diagrams for clarity.